Fuzzy Deep Learning based Urban Traffic Incident Detection

Chaimae EL HATRI
Email: chaimae.elhatri@usmba.ac.ma

Jaouad BOUMHIDI
Email: jaouad.boumhidi@usmba.ac.ma

Computer Science Department
Sidi Mohamed Ben AbdEllah University, LIIAN laboratory
Faculty of Sciences Dhar-Mahraz, Fez 30000, MOROCCO

***Abstract*— Traffic incident detection (TID) is an important part of any modern traffic control because it offers an opportunity to maximise road system performance. For the complexity and the nonlinear characteristics of traffic incidents, this paper proposes a novel fuzzy deep learning based TID method which considers the spatial and temporal correlations of traffic flow inherently. Parameters of the deep network are initialized using a Stacked Auto-Encoder (SAE) model following a layer by layer pre-training procedure. To conduct the fine tuning step, the back-propagation algorithm is used to precisely adjust the parameters in the deep network. Fuzzy logic is employed to control the learning parameters where the objective is to reduce the possibility of overshooting during the learning process, increase the convergence speed and minimize the error. To find the best architecture of the deep network, we used a separate validation set to evaluate different architectures generated randomly based on the Mean Squared Error (MSE). Simulation results show that the proposed incident detection method has many advantages such as higher detection rate and lower false alarm rate.**

Keywords — Automatic Traffic Incident Detection; Fuzzy Deep Learning; Deep Artificial Neural Network; Stacked Auto-Encoder Model; Back-Propagation Algorithm; Fuzzy Logic Controller

I. INTRODUCTION

The congestion is major problem on the road in the urban cities. One of the important parts of intelligent transportation systems is incident detection. Traffic incidents are the main cause of congestion, which subsequently increases travel delay and fuel consumption in major cities. Traffic incidents usually cannot be predicted which poses great challenge to traffic management centers. Therefore, early detection of incidents reduces the delay experienced by road users, fuel consumptions, gas emissions, and the probability of other collisions, as well as improves road safety and real-time traffic control [1]. This paper focuses on developing an intelligent system able to determine the presence of an incident by using real-time data. The objective is to help the officer manage and take action to clear the roadway as early as possible and to ensure that traffic return to normal safety conditions.

The problem of TID can be regarded as a task of classification, to determine whether or not an incident happens according to data gathered from traffic flow. Various automatic incident detection techniques have been released to address this problem, such as Support Vector Machine (SVM) [2], Fuzzy Logic (FL) [3], and Back-Propagation Neural Network (BPNN) [4]. All of the works cited suffer from drawbacks. The limitation of SVM comes from the choice of kernel function, the determination and tuning of several parameters. FL is based on human knowledge by determining fuzzy rules which are often set manually by experts. Thus, the detection performance is affected by subjective decisions. BPNN suffers from slow convergence and the possibility of getting caught in the local minimum. In any event, it is difficult to say that one method is clearly superior ever other methods in any situation. One reason for this is that the proposed models are developed with a small amount of specific data. The accuracy of TID methods is dependent of the traffic flow features embedded in the collected spatiotemporal traffic data. In this paper, we propose a novel TID method which considers the spatial and temporal correlations of traffic flow inherently. In general, the use of intelligent classification offers helpful techniques to deal with this kind of problem. Artificial intelligence refers to a set of procedures that apply reasoning and uncertainty in complex decision making and data analysis processes [5]. Among artificial intelligence methods, Artificial Neural Networks (ANNs) have been widely used in various research areas to solve problems including classification, traffic incident detection, function approximation, optimization, prediction, and so on [6-7]. Some of the studies confirmed that neural network models can provide fast and reliable incident detection for several reasons. First, automatic incident detection can be cast as a pattern recognition problem. Neural networks are known to solve pattern recognition problems effectively [8]. Second, neural networks are suitable for solving the problem when there is no mathematical model or explicit rules.

There are a variety of types of ANNs. Traditional process neural network is usually limited in the structure of single hidden layer. ANN with multiple layers was examined not as effective as ANN with single hidden layer in many applications. It is because the structure of process neural network would be very complex when extended to multiple hidden layers. However, for complex detection systems with rich amount of data, one single hidden layer usually would be not enough in describing complicated relations between inputs and outputs. Complexity theory of circuits strongly suggests that deep architectures can be much more efficient than shallow architectures, in terms of computational elements and

978-1-5090-4062-9/17/$31.00 ©2017 IEEE

parameters required to represent some functions. Deep learning is a type of machine learning method. It has been introduced with the objective of moving machine learning closer to one of its original goals: artificial intelligence. The concept of deep learning is derived from neural network research, therefore deep learning regarded as a new generation of neural networks [9]. It use multiple-layer architectures or deep architecture to extract inherent features in data from the lowest level to the highest level, and they can discover huge amounts of structure in the data. Recently, deep learning had attracted a lot of attention from both academic and industrial communities. It has been applied with success in classification tasks, natural language processing, object detection, traffic data prediction and so on [10-13].

As traffic incident is complicated in nature, deep learning algorithms can represent traffic features without prior knowledge, which may have good performance for traffic incident detection. In this paper, we propose a novel fuzzy deep learning based incident detection method. The parameters of the deep neural network are initialized using a stacked auto-encoder model following a layer by layer pre-training procedure to represent traffic flow features for incident detection. To conduct the fine tuning step, we used the back-propagation algorithm to precisely adjust the parameters and a fuzzy logic control to adaptively vary the learning parameters where the objective is to reduce the possibility of overshooting during the learning process and help the deep network get out of a local minimum. The performance of a deep neural network model is very sensitive to the proper selection of network architecture; the number of hidden layers and the numbers of nodes in each layer. To find the best architecture, we used a separate validation set to evaluate different architectures based on the mean squared error. The proposed method is inspired from [14]. The simulation results show that the fuzzy deep learning based TID method has the advantages of high detection and classification rates and low false alarm rate. The rest of the paper is organized as follows: fuzzy deep learning based approach is presented in Section 2. Network configuration and measures to evaluate detection performance are discussed in Section 3. Simulation results and analysis are given in Section 4. Finally, the paper is concluded in Section 5.

II. METHODOLOGY

A. The Structure of Stacked Auto-Encoders

A SAE is build by stacking a series of auto-encoders. An Auto-Encoder (AE) is a neural network which has the same number of nodes in the input and output layers. The simplest form is a neural network with only one hidden layer. Its structure is shown in Figure 1. Given a set of training samples $\{x^{(1)}, x^{(2)}, x^{(3)}, ..\}$, where $x^{(i)} \in R^d$, an AE first encodes an input $x^{(i)}$ to a hidden representation $y(x^{(i)})$ computed by Equation (1), and then it decodes representation $y(x^{(i)})$ back into a reconstruction $z(x^{(i)})$ computed by Equation (2) [15].

$$y(x) = f(W_1.x + b_1) \quad (1)$$

$$z(x) = g(W_2.y(x) + b_2) \quad (2)$$

where W_1 is a weight matrix, b_1 is an encoding bias vector, W_2 is a decoding matrix, and b_2 is a decoding bias vector. $f(x)$ and $g(x)$ are the activation functions where logistic sigmoid function is applied in this paper.

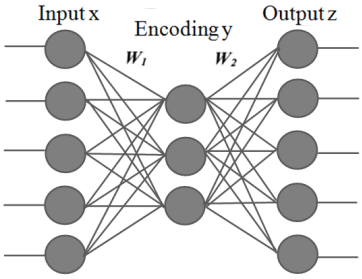

Fig. 1. The structure of an one-hidden-layer auto-encoder.

The auto-encoder can be trained using the conventional back-propagation algorithm. Training of auto-encoder takes each input sample itself as its label. The target is to minimize the reconstruction error $E(X, Z)$ defined by Equation (3).

$$E(X, Z) = \frac{1}{2} \sum_{i=1}^{N} ||x^i - z^i||^2 \quad (3)$$

where X is the set of N input samples $\{x^{(1)}, x^{(2)}, .., x^{(N)}\}$, and Z is the set of corresponding reconstructed output $\{z^{(1)}, z^{(2)}, .., z^{(N)}\}$. And the notation $\|.\|$ represents the 2-norm of a vector.

A SAE model is created by stacking auto-encoders to form a deep neural network by taking the output of the auto-encoders found on the layer below as the input of the current layer. Considering a SAE with 3 layers as shown in Figure 2, and given a set of input samples, the training of an SAE is straightforward. The first layer is trained as an auto-encoder, with the training set as inputs such that ${W_1}^1$and ${W_2}^1$are obtained. Then for each *i (i=2,3)* AE, the encoding of the first AE y^{i-1} is taken as the input to train this AE such that ${W_1}^i$and ${W_2}^i$are obtained. In this way the training of multiple auto-encoders is completed.

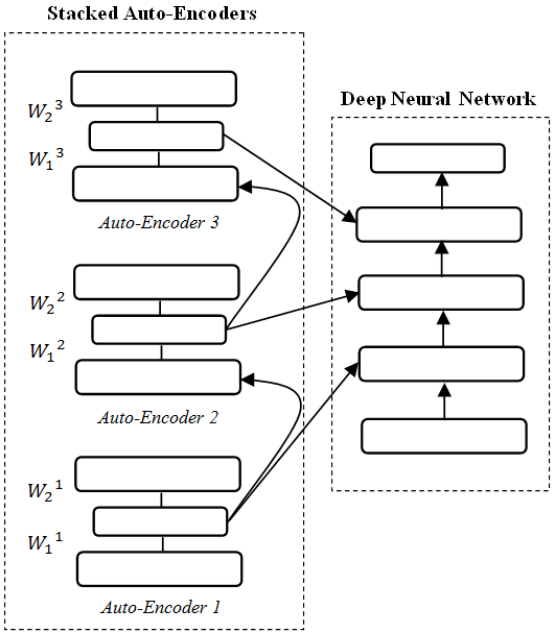

Fig. 2. The structure of a Stacked Auto-Encoders.

B. Back-Propagation Algorithm

Fine tuning is a strategy that is commonly found in deep learning. It can be used to greatly improve the performance of stacked auto-encoders. From a high level perspective, fine tuning treats all layers of SAE as a single model, so that in one iteration, we are improving upon all the weights. As the back-propagation algorithm which is based on descent gradient technique can be extended to apply for an arbitrary number of layers, we can actually use this algorithm on stacked auto-encoders of arbitrary depth. In this work, to adapt the connections weights in order to obtain minimal difference between the network output and the desired output, we used the back-propagation algorithm. It was developed by Paul Werbos in 1974. BP algorithm is quite simple; output of NN is evaluated against desired output. If results are not satisfactory, connection between layers are modified and process is repeated again until error is small enough. The objective of BP method is adapting synaptic weights in order to minimize an error function. The approach most commonly used for the minimization of the error function is based on the gradient method. The error function is defined by Equation (4).

$$E(t) = \frac{1}{2}|e(t)|^2 \tag{4}$$

Where $e(t)$ is the error value, i.e. the difference between the output and the estimated output. This difference is used to change the connection weights between neuron in the network according to Equation (5).

$$\Delta w_j(t+1) = \eta\left(\frac{\partial E(t)}{\partial w_j}\right) + \alpha\, \Delta w_j(t) \tag{5}$$

Where η is the learning rate parameter. The positive constant α is a momentum factor. Neural networks are sensitive to the selection of learning rate and momentum. Learning rate can limit or expand the extent of weight adjustment in a learning cycle. A higher learning rate can make the network unstable and can cripple the network prediction ability. In the contrary if the learning rate is low, the training time of the network is increased. In the other hand, low momentum causes weight oscillations and instability and thus preventing the network from learning. High momentum can cripple the network adaptability. During the middle of training, when steep slope occurs, a small momentum value is recommended. However, during the end of training, large momentum value is desirable. Therefore, for the best results, it is necessary that the learning parameters should be adjusted adaptively during the learning process, since no single value is optimal for all dimensions. In the next part, the implementation of fuzzy control system to adaptively determine the learning rate and momentum value is discussed.

C. Fuzzy Logic Controlled Deep Neural Network

A fuzzy logic control is a technique useful in representing human knowledge in a specific domain of application and in reasoning with that knowledge to make useful inferences or actions [16]. A fuzzy logic control system consists of four components. A fuzzifier converts data into fuzzy data or Membership Functions (MFs). The fuzzy rule base contains the relations between the input and output. The fuzzy inference process combines MFs with the control rules to derive the fuzzy output, and the deffuzifier converts the fuzzy numbers back to a crisp value.

There are two reasons that fuzzy logic systems are preferred: fuzzy systems are suitable for uncertain or approximate reasoning and they allow decision making with estimated values under incomplete or uncertain information. By employing a fuzzy control system to adaptively adjust the learning parameters of the neural network according to the MSE error, we can reduce the possibility of overshooting during the learning process and help the network get out of a local minimum. There are four parameters used to create the rules for the fuzzy logic control system; the relative error (RE), change in relative error (CRE), sign change in error (SC) and cumulative sum of sign change in error (CSC). The four parameters are described as follows:

$$\begin{cases} RE(t) = E(t) - E(t-1) \\ CRE = RE(t) - RE(t-1) \\ SC(t) = 1 - ||\frac{1}{2}[sign(RE(t-1) + sign\big(RE(t)\big)|| \\ CSC = \sum_{m=t-4}^{t} SC(m) \end{cases} \tag{6}$$

For simplicity, we assume that the fuzzy logic system contains two inputs RE and CRE, and two output; the change in the learning parameter $\Delta\eta$ and change in the momentum value $\Delta\alpha$. The range of RE and CRE is 0-1. The linguistic values of RE, CRE and $\Delta\eta$ are NL, NS, ZE, PS and PL. NL represents a "Negative Large" value, NS is "Negative Small", ZE is "Zero", PS is "Positive Small" and PL is "Positive Large". The change in the momentum value is small enough to avoid overcorrection. Table 1 and Table 2 describe the matrix representations of the rule bases for $\Delta\eta$ and $\Delta\alpha$, respectively. Two of the fuzzy IF-THEN rules that are employed are listed:

Rule1: IF RE is small AND CSC is less or equal to two, THEN the value of η and α should be increased.

Rule1: IF CSC is larger than three, THEN the value of the value of η and α should be decreased regardless the value of RE and CRE.

TABLE I. DECISION TABLE FOR $\Delta\eta$ WHEN CSC ≤ 2.

CRE	RE				
	NL	**NS**	**ZE**	**PS**	**PL**
NL	NS	NS	NS	NS	NS
NS	NS	ZE	PS	ZE	NS
ZE	ZE	PS	ZE	NS	ZE
PS	NS	ZE	PS	ZE	NS
PL	NS	NS	NS	NS	NS

TABLE II. DECISION TABLE FOR $\Delta\boldsymbol{\alpha}$ WHEN CSC ≤ 2.

CRE	RE				
	NL	**NS**	**ZE**	**PS**	**PL**
NL	-0.01	-0.01	0	0	0
NS	-0.01	0	0	0	0
ZE	0	0.01	0.01	0.01	0
PS	0	0	0	0	-0.01
PL	0	0	0	-0.01	-0.01

D. Fuzzy Deep Neural Network Training

Deep multi-layer neural networks have many levels of non-linearity allowing them to compactly represent highly non-linear and highly varying functions. The training phase of deep neural network contains two major steps of parameter initialization and fine tuning. The initialization step is critical in deep learning because the whole learning system is not convex. A better initialization strategy may help the neural network to converge to a good local minimum more efficiently. In this paper, a SAE model is introduced. The fine tuning step allows to precisely adjusting the parameters in the neural network in a supervised way to enhance the discriminate ability of the final feature presentation. To conduct this step, we propose to use the back-propagation method with gradient based optimization technique and an on-line fuzzy logic controller in the role of supervised to adapt the learning parameters based on the mean squared error generated by the deep neural network. The training procedure can be summarized by Algorithm 1.

Algorithm 1: Fuzzy Deep Network Training Algorithm

Given training sample X and the structure of the deep neural network.

Step 1: Parameters Initialization

1. Train the first layer as an auto-encoder with the training sets as the input using the back-propagation algorithm.

2. Train the second layer as an auto-encoder taking the first layer's output as input.

3. Iterate as in 2 for the desired number of layers.

Step 2: Fine-Tuning

4. Use the output of the last layer as the input for the classification layer, and initialize its parameters by supervised training.

5. Fine-tune the parameters of all layers with the Back-Propagation method in a supervised way using fuzzy logic controller to adaptively adjust the learning parameters.

III. Network Configuration and Performance Measures

Traffic incidents are the main cause of traffic congestion, which subsequently increases travel delay and fuel consumption in major cities. TID became very important topic in traffic management systems. Detection of incident will avoid future traffic incidents and will help authorities to make road segment available for traffic again. This article focuses on the application of a fuzzy deep learning method for TID. To improve the efficiency of the proposed method in detecting traffic incidents, we used the microscopic traffic simulator SUMO (Simulator for Urban Mobility). Our traffic network is shown in Figure 3. It contains twenty-five four way intersections equipped with traffic signals and more than 100 edges. Each edge contains three lanes in each direction. Each lane has two detectors, one at the entry to collect upstream traffic data and the other at the exit to collect downstream traffic data. During the simulation, more than 1000 vehicles are generated by uniform distribution over 20 input sections.

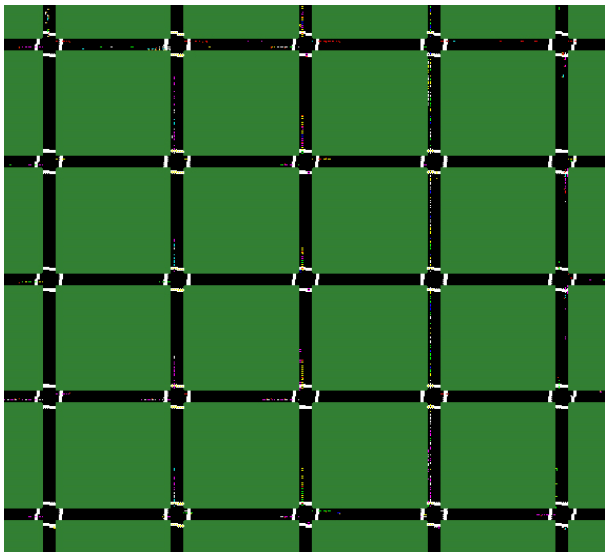

Fig. 3. Network configuration composed of twenty-five intersections.

To train and validate our proposed model, sufficient data that includes different incident patterns under a variety of flow conditions has to be acquired. There are a few options in SUMO to simulate traffic incident such as stopping a car for a fixed period of time, by defining a point along its route when it should halt and for how long. We decided to employ this option since it is the easiest to implement. We introduced 30 incidents and we aggregated the data in 100-second intervals. Our dataset contained traffic information including mean speed, occupancy rate, current traffic flow X^t and the traffic flow at previous time intervals, i.e., $X^{t-1}, X^{t-2}, \ldots, X^{t-r}$. Therefore, the proposed model accounts for the temporal correlations of traffic flow. The model can be build from the perspective of a transportation network considering also the spatial correlations of traffic flow. Thus, we collected the data from all edges of our network. The dataset was captured by 360 pairs of inductive loop detectors. It contains 15000 samples divided into three distinct sets called training, testing and validation sets. The training set with 60% sample is used to train the deep network model to get optimal parameters. The testing set with 20% samples is used to predict future performance of the network. A final check of the performance of the trained network is made using validation set with 20% samples. The sets are selected randomly from the full dataset. It is the simplest method for dividing the data which usually produces good results.

The performance of a deep neural network model is very sensitive to the proper selection of network architecture. To find the best deep neural network for a given dataset, we need to decide such specific details as how many neurons and layers we want to use and how many outputs the network should have. Different combination of number of layers and number of nodes in each layer makes different architectures of a deep network. The basic architecture selection process proposed in this paper is described in Algorithm 2. We randomly generate a set of architectures. Each architecture is trained and the MSE is computed for the training set and the validation set. The goal is to find a neural network architecture with least mean of $MSE_{training}$ and $MSE_{validation}$. The MSE of the training

set $MSE_{training}$, the MSE of the testing set $MSE_{testing}$ and the MSE of the validation set $MSE_{validatin}$ are defined by the following equations:

$$MSE_{training} = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} (d_i - o_i)^2 \quad (7)$$

$$MSE_{testing} = \frac{1}{N_{te}} \sum_{i=1}^{N_{te}} (d_i - o_i)^2 \quad (8)$$

$$MSE_{validation} = \frac{1}{N_{va}} \sum_{i=1}^{N_{va}} (d_i - o_i)^2 \quad (9)$$

Where the MSE is the mean $(\frac{1}{N} \sum_{i=1}^{N})$ of the square of the errors $(d_i - o_i)^2$. N_{tr} is the number of training data, N_{te} is the number of testing data and N_{va} is the number of validating data. d_i is the desired output and o_i is the actual output produced by the neural network.

Algorithm 2: Architecture Selection Process

1. Gather Data
2. Divide data into three sets
3. **For all** Candidate architectures **Do**
4. Train network with training set
5. $MSE_{training}$ ← MSE of training set
6. $MSE_{validation}$ ← MSE of validation set
7. **End For**
8. Choose network with minimum mean of $MSE_{training}$ and $MSE_{validation}$

There are numerous measures to evaluate the detection performance such as Detection Rate (DR), False Alarm Rate (FAR), Mean Time to Detection (MTTD) and Classification Rate (CR). DR is defined as the percentage of incident cases detected correctly by the system. FAR is one of the main parameters for evaluating incident detection systems. It is an index to represent the rate at which the non-incident cases are falsely classified as incident cases, thus raising a false alarm. MTTD is computed as the average length of time between the start of the incident and the time the alarm is initiated. The first correct alarm declared for a single incident is used for computing MTTD. CR is computed as the percentage of classified objects including both incident and non-incident objects, by the model of the total number of testing objects. The measures mentioned above are proposed as follows:

$$DR = \frac{number\ of\ incident\ detected}{total\ number\ of\ incident\ cases} * 100 \quad (10)$$

$$FAR = \frac{number\ of\ false\ alarm\ cases}{total\ number\ of\ non-incident\ instances} * 100 \quad (11)$$

$$MTTD = \frac{t_1 + t_2 + \cdots + t_m}{m} \quad (12)$$

$$CR = \frac{TP + TN}{P + N} * 100 \quad (13)$$

Where t_i is the time delay between the occurrence of the incident and its detection. m is the number of the incidents detected. P is the number of positive objects indicating an incident state and N is the number of negative objects indicating an incident free state. TP and TN are numbers of true positive and true negative respectively.

IV. Simulation and Results

Our simulation is divided basically into three parts. We determine the optimal structure of the deep network in the first part. In the second part, we compare four models: process neural network with single hidden layer (SLNN), process neural network with multiple hidden layers (MLNN), deep process neural network (DNN) and fuzzy deep process neural network (FDNN). Notice that SLNN and MLNN are training in supervised way while DNN and FDNN are training in unsupervised way. The structure of MLNN, DNN and FDNN are exactly the same. The only difference is the training algorithm. MLNN is training with traditional learning method. In the last part, the comparison is made between the DNN and FDNN in term of speed of convergence.

One important issue in neural network is selecting appropriate structure. Different combination of number of layers and number of neurons in each layer makes different structures of a deep network. We generated 5 structures randomly. We limited the number of hidden layers between 3 and 7, and the number of neurons in each layer between 3 and 20. Simulation results are reported in Table 3. The structure with minimum mean of $MSE_{training}$ and $MSE_{validation}$, is determined as the optimal structure. The minimum mean of $MSE_{training}$ and $MSE_{validation}$ is (0.16 + 0.14)/2=0.15. Thus, the optimal structure is [11, 12, 10, 14], which means the deep network has four AEs stacked whose number of neurons are 11, 12, 10 and 14 respectively.

TABLE III. Comparaison between different structures.

# of AEs	Structure	$MSE_{training}$	$MSE_{validation}$
3	[8,10,15]	0.54	0.85
4	[11,12,10,14]	**0.16**	**0.14**
5	[19,5,20,9,11]	0.28	0.23
6	[9,8,15,13,18,12]	0.33	0.45
7	[16,11,17,8,5,13,9]	0.37	0.57

To evaluate the TID method, we used the performance measures. DR and FAR quantify the effectiveness of an algorithm while the MTTD and CR reflect the efficiency of the algorithm. The MSE assesses the quality of our estimator. Another term of comparison is training time (TT). It can calculate the time consumed by the method to reach an optimal classifier. Tables 4 and 5 summarize the testing performance of SLNN, MLNN, DNN and FDNN. It is evident that FDNN showing high detection rate, low false alarm rate, high classification rate, least $MSE_{training}$ and least $MSE_{testing}$, performs better than SLNN, MLNN and DNN. SLNN is the fastest since it is with the simplest structure of single hidden layer. Training time of DNN compared to MLNN does not increase a lot. One reason maybe that time complexity of training a process neural network and training a deep auto-encoder is similar. DNN uses fixed learning rate and fixed momentum term. 0.5 is set as the value of η and α. We can conclude that FDNN is far way better that DNN; the previous can converge in 275 seconds better than DNN which takes 383 seconds. FDNN showed a 28.19% average improvement in training time by employing a fuzzy logic controller to adaptively vary the learning parameters.

TABLE IV. COMPARAISON BETWEEN SLNN, MLNN, DNN AND FDNN IN TERMS OF INDICES OF PERFORMANCE.

	Detection Rate	False Alarm Rate	Mean Time to Detection	Classification Rate
SLNN	80.24%	0.33%	161.55s	77.54%
MLNN	87.58%	0.32%	**155.02s**	83.02%
DNN	92.89%	0.23%	188.52s	89.79%
FDNN	**98.23%**	**0.24%**	192.44s	**91.47%**

TABLE V. COMPARAISON BETWEEN SLNN, MLNN, DNN AND FDNN IN TERMS OF MEAN SQUARED ERROR AND TRAINING TIME.

	Training Time	$MSE_{training}$	$MSE_{testing}$
SLNN	**122s**	0.34	0.35
MLNN	321s	0.31	0.32
DNN	383s	0.19	0.18
FDNN	275s	**0.16**	**0.13**

Finally, we make comparison on converting speed and effect. We have provided training details in Figure 4. FDNN is faster on converging than DNN, MLNN and SLNN; FDNN could reach convergence in less than 7 iterations and DNN in 12 iterations, while MLNN requires about 15 iterations and SLNN needs about 20 iterations. From the results, we could find that DNN is effective in traffic incident detection. More layers in process neural network could capture better feature representation. Training algorithm based on fuzzy deep learning is also much more effective than traditional deep learning. By employing a fuzzy logic system to adjust the learning parameters, the fuzzy deep learning method could avoid sticking in local minimum effectively, increase the convergence speed and minimize the error.

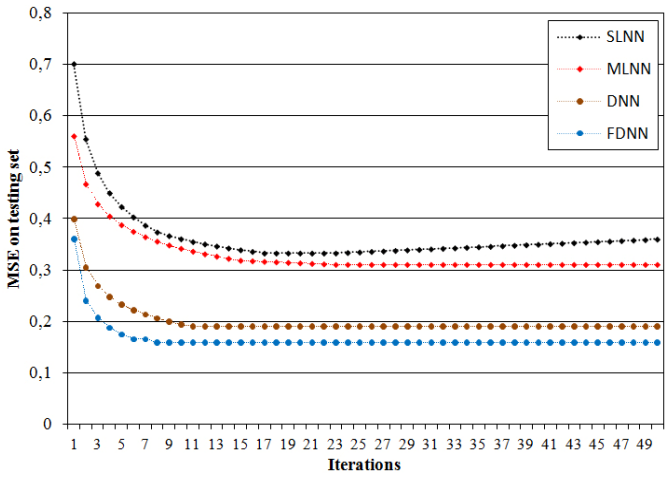

Fig. 4. Comparison on converting speed and effect.

V. CONCLUSIONS

This paper proposes a novel fuzzy deep learning approach with a stacked auto-encoders model for automatic TID problem. We applied unsupervised learning algorithm to pre-train the deep network, and then, we did the fine-tuning process to update the parameters of network in a supervised way taking training data as the input and their labels as the target output, using a back-propagation algorithm. The back-propagation algorithm suffers from some drawbacks. First, slow convergence rate and second the possibility of settling into a local minimum. To avoid these issues, fuzzy logic system is employed to control the learning parameters. The performance of the proposed method was evaluated in terms of detection rate, false alarm rate, mean time to detection, classification rate, training time and mean squared error. Simulation results indicate that FDNN is efficient and effective learning technique for TID, surpass SLNN and MLNN in terms of indices of performance and DNN in term of speed of convergence. FDNN minimize the error and reduce the training time by 28.19% comparing to DNN. The obtained results are very satisfactory and show the superiority of the FDNN model.

REFERENCES

[1] C. El Hatri, M. Tahifa, and J. Boumhidi, "Extreme Learning Machine-Based Traffic Incidents Detection with Domain Adaptation Transfer Learning," Journal of Intelligent Systems, October 2016.

[2] F. Yuan and R. L. Cheu, "Incident detection using support vector machines," Transport. Res. Pt. C Emerg. Technol. 1, 2003, pp. 309–328.

[3] R. Rossi, M. Gastaldi, G. Gecchele and V. Barbaro, "Fuzzy logic-based incident detection system using loop detectors data," Transport. Res. Proc. 10, 2015, pp. 266–275.

[4] J. Lu, S. Chen, W. Wang and H. van Zuylen, "A hybrid model of partial least squares and neural network for traffic incident detection," Expert Syst. Appl. 39, 2012, pp. 4775–4784.

[5] R. Rossi, M. Gastaldi, G. Gecchele and V. Barbaro, "Fuzzy logic-based incident detection system using loop detectors data," Transportation Research Procedia10, 2015, 266–275.

[6] M. Chakraborty, "Artificial neural network for performance modeling and optimization of CMOS analog circuits," International Journal of Computer Applications, vol. 58, no. 18, 2012, pp. 6–12.

[7] M. B. W. de Oliveira and A. de Almeida Neto, "Optimization of Traffic Lights Timing based on Artificial Neural Networks," 2014 IEEE 17th International Conference on Intelligent Transportations Systems (ITSC), October 8-11, 2014.

[8] J. Kumar Basu, D. Bhattacharyya and T. Kim, "Use of Artificial Neural Network in Pattern Recognition," International Journal of Software Engineering and its Applications, Vol. 4, N. 2, April 2010.

[9] M. Chakraborty, "Artificial neural network for performance modeling and optimization of CMOS analog circuits," International Journal of Computer Applications, vol. 58, no. 18, 2012, pp. 6–12.

[10] L. Deng and D. Yu, "Deep Learning: Methods and Applications," Foundations and Trends in Signal Processing, Vol. 7, pp. 197-387, 2013.

[11] H.-C. Shin, M. R. Orton, D. J. Collins, S. J. Doran, and M. O. Leach, "Stacked autoencoders for unsupervised feature learning and multiple organ detection in a pilot study using 4D patient data," IEEE Trans. Pattern Anal. Mach. Intell., vol. 35, no. 8, Aug. 2013, pp. 1930–1943.

[12] Y. DENG, Z. Ren, Y. kong, F. Bao and Q. Dai, "A Hierarchical Fused Fuzzy Deep Neural Network for Data Classification," IEEE Trasactions on Fuzzy Systems, June 2016.

[13] Y. Lv, Y. Duan, W. Kang, Z. Li, and F. Wang., "Traffic flow prediction with big data: A deep learning approach," ITS, 2015, pp. 865–873.

[14] J. L. Wright and M. Manic, "Neural Network Architecture Selection Analysis with Application to Cryptography Location," WCCI IEEE World Congress on Computational Intelligence, 2010.

[15] W. huand, H. Hong, G. Song and K. Xie, "Deep Process Neural Network for Temporal Deep Learning," International Joint Conference on Neural Networks, 2014.

[16] U.F. Eze, I. Emmanuel and E. Stephen, "Fuzzy Logic Model for Traffic Congestion," Journal of Mobile Computing and Application, 2014, pp. 15-20.